

ML Architecture Deep Dive — Ball Mill PSI200 Soft Sensor

Purpose of this document. We have a noisy plant historian, a motif-extraction stage, and a two-stage XGBoost cascade predicting **PSI200**. Before we change code, this document explains *why* each part of the current design behaves the way it does, what the underlying ML concept is, and what the realistic alternatives are — with their trade-offs — so we can pick deliberately instead of guessing.

Each topic follows the same structure:

- **What the code does today** (with file references)
 - **The ML concept, explained simply**
 - **Why it matters here** (concrete numbers/examples from our plant)
 - **The candidate paths** (A / B / C ...) with pros, cons, cost
 - **Recommendation**
-

Table of Contents

1. [Topic 1 — Acausal Smoothing and Target Leakage](#)
 2. [Topic 2 — Label Granularity and Effective Sample Size](#)
 3. [Topic 3 — Dead Time and Temporal Alignment](#)
 4. [Topic 4 — Motif Selection vs. Steady-State Detection](#)
 5. [Topic 5 — Variable Taxonomy \(MV / CV / DV\) and Model Invertibility](#)
 6. [Topic 6 — Cascade Coupling and Error Compounding](#)
 7. [Topic 7 — Validation Strategy Under Autocorrelation](#)
 8. [Topic 8 — Hyperparameter Search and the Noise Floor](#)
 9. [Topic 9 — Extrapolation, Physics Constraints, and Hybrid Models](#)
 10. [Topic 10 — Uncertainty Quantification and Out-of-Domain Detection](#)
 11. [Topic 11 — Data Leverage: Multi-Mill Pooling and Semi-Supervised Use](#)
 12. [Decision Matrix and Recommended Path](#)
-

Topic 1 — Acausal Smoothing and Target Leakage

What the code does today

`db/db_connector.py` has a `no_interpolation` switch, and `data_preparation` passes `True`. But for **mill data** the flag is overridden and hardcoded to `False`:

```
# db/db_connector.py:490
processed_mill_data = self.process_dataframe(
    mill_data, start_date, end_date, resample_freq, no_interpolation=False
)
```

That routes mill data into the interpolating branch, which ends with:

```
# db/db_connector.py:356
df_processed = df_processed.rolling(window=window_size, min_periods=1,
center=True).mean()
```

`window_size = 15, center=True.`

The ML concept, explained simply

Data leakage is when information that would *not* be available at prediction time sneaks into the training features. The model learns to use it, scores brilliantly offline, and then collapses in production.

A **centered** rolling mean is the classic acausal filter. For the row at 10:00, it averages 09:53 → 10:07. Seven of those fifteen minutes are **in the future**.

Simple example. Suppose **Ore** is a step: 150 t/h until 10:00, then 180 t/h.

Time	Raw Ore	Causal mean (past 15)	Centered mean (±7)
09:55	150	150	150
09:58	150	150	160
10:00	180	152	165

Time	Raw Ore	Causal mean (past 15)	Centered mean (± 7)
10:03	180	162	176

Look at 09:58. The centered value already reads 160 — the feature **announces a step change that has not happened yet**. If **PSI200** responds to that step, the model discovers a feature that predicts the target before the cause occurred. Test R^2 goes up. Real-world value: zero, because at 09:58 in production you cannot compute that number.

Why it matters here

Two compounding effects:

1. **Optimistic metrics.** Every reported R^2 in **training_results.json** is inflated by an unknown amount. We cannot tell whether "the model is good" until this is removed.
2. **Undeployable features.** At inference you would have to wait 7 minutes to build each feature vector, or substitute a causal filter — at which point the model sees a feature distribution it was never trained on. That is **train/serve skew**, and it typically costs more than the leakage gained.

There is a third, subtler issue: smoothing is applied to **all numeric columns including the target**. A smoothed target is easier to predict than a raw one (you removed the noise you were supposed to fail on). That inflates R^2 again, independently of the centering.

The candidate paths

Path A — Causal rolling mean. `rolling(window=15, min_periods=1).mean()`
(drop **center**).

- Pro: one-word change; features become reproducible online.
- Con: introduces a lag of ~7 min into every feature (a rolling mean's group delay is $(w-1)/2$). Since we also need to *add* deliberate lags (Topic 3), this delay is not harmful — but it must be accounted for, not stacked blindly.

Path B — Exponentially weighted mean (EWMA). `ewm(halflife='5min').mean()`

- Pro: causal, smaller group delay for the same noise reduction, no hard window boundary, naturally handles irregular sampling.
- Pro: exactly what a DCS/PLC would compute online — trivially reproducible in production.
- Con: one more hyperparameter (half-life) to select.

Path C — Robust causal filter (rolling median / Hampel).

- Pro: the historian is *noisy with spikes* (dropouts, frozen sensors, transmitter glitches). A mean is dragged by a single -9999; a median ignores it.
- Pro: complements the existing `filter_data_adaptive` (rolling median + MAD) already in `core/data_loader.py`.
- Con: slower; non-linear, so it can flatten genuine fast transients.

Path D — No smoothing in the loader; smooth as an explicit feature step. Keep raw 1-min data in `initial_data.csv`, and generate `Ore_ewm5`, `Ore_ewm30`, `Ore_std15` as *named features* in the modeling layer.

- Pro: **This is the cleanest architecture.** Smoothing becomes a modeling decision, visible and tunable, not a hidden side effect buried in a DB connector. It also lets the model use *multiple* timescales at once, which is genuinely informative (a 5-min mean and a 60-min mean tell different stories).
- Pro: makes the raw data available for the variability-based motif constraints, which are currently operating on **already-smoothed** data — meaning our "stable vs. varying" CV thresholds (`cv <= 1%`, `cv >= 0.08%`) are measuring the smoother as much as the process.
- Con: largest refactor; `initial_data.csv` and every downstream threshold need re-tuning.

Recommendation

Path D as the destination, Path B as the immediate step. Change the mill-data call to be causal (EWMA) right now so we can re-baseline honestly; then, in a second pass, move smoothing out of the connector into an explicit multi-timescale feature stage. Also stop smoothing the target column — smooth features only.

Expected effect: reported R^2 will *drop*. That is success, not regression. Record the before/after in a table so we never re-litigate this.

Topic 2 — Label Granularity and Effective Sample Size

What the code does today

Ore-quality data (which carries `Class_15`, `Daiki`, `FE`, and — depending on the source table — the lab `PSI200`) goes through the forward-fill branch:

```
# db/db_connector.py:337-343
df_resampled = df_processed[numeric_cols].resample(resample_freq).ffill()
df_processed = df_resampled.ffill()
df_processed = df_processed.bfill()
```

So one lab value becomes N identical 1-minute rows. The model then trains and is scored on all N.

The ML concept, explained simply

Effective sample size (ESS) is how many *independent* observations you actually have — not how many rows are in the DataFrame.

Concrete example. Say a lab sample is taken every 8-hour shift, and we have 6 months of data:

- Rows after ffill to 1-min: $180 \text{ days} \times 1440 = 259,200$
- Actual distinct labels: $180 \text{ days} \times 3 \text{ shifts} = 540$

The model reports metrics over 259,200 samples, but the target contains **540 numbers**. It is a 480× replication factor.

Three separate things go wrong:

1. **Metric inflation via replication.** With `test_size=0.2`, the test set has ~52,000 rows but only ~108 distinct labels. R^2 computed over the replicated rows looks stable and precise; the true uncertainty is that of a 108-sample estimate. The confidence interval on R^2 is roughly 5× wider than it appears.
2. **Fold contamination.** `TimeSeriesSplit` splits by row index. A shift's 480 replicas are contiguous, so *most* stay together — but every fold boundary cuts through

one shift, putting identical labels with near-identical features on both sides. Worse, after motif extraction the rows are no longer contiguous in original time, so replicas of the same lab value can be scattered.

3. **Loss-weighting distortion.** A shift with a long steady run contributes 480 rows; a shift with a short run after filtering might contribute 40. The model implicitly weights the first shift 12× more, purely because of duration, not information content.

Why it matters here

This is, in my judgement, the **single largest source of the gap between offline metrics and plant reality** — bigger even than Topic 1. A model that has genuinely seen 540 target values cannot support a 243-point hyperparameter grid, cannot support 12 input features without regularization, and definitely cannot support claims of $R^2 = 0.9$.

Note this also interacts with Topic 1: the filled target is *piecewise constant*, and a centered rolling mean over a piecewise-constant signal creates ramps that straddle the change points — literally interpolating the future label into the present row.

The candidate paths

Path A — Do nothing, but report ESS-corrected metrics. Keep training as-is, but compute test metrics **grouped by lab sample** (average predictions within each sample, then score against the 108 distinct labels).

- Pro: zero pipeline change; immediately gives an honest number.
- Pro: excellent as a *diagnostic* to quantify how bad the problem is.
- Con: doesn't fix training, only measurement.

Path B — Sample weighting. Give each row weight `1 / n_replicas_in_its_group`.

- Pro: cheap; XGBoost supports `sample_weight` natively.
- Pro: every shift contributes equally regardless of duration.
- Con: the model still sees 259,200 near-duplicate rows — training is slow and trees still overfit to the *feature* variation within a constant-label block, learning "noise → label".

Path C — Aggregate to the label's native resolution. ★ For each lab sample at time t , build **one** training row from the residence window $[t - \text{lag} - w, t - \text{lag}]$:

Aggregate	Example features
mean	Ore_mean , WaterMill_mean , DensityHC_mean
std	Ore_std (captures how steady the run was)
slope	DensityHC_slope (captures whether the circuit was drifting)
min/max	PressureHC_max (captures upsets)
last	Ore_last (the value nearest the sample)

Dataset becomes 540 rows $\times$ ~30 features, target = 540 real lab values.

- Pro: **structurally eliminates** replication, fold contamination, and duration weighting.
- Pro: matches the physics — a lab sample reflects an *interval* of operation, not a minute.
- Pro: the std/slope aggregates encode "was this steady?" as a **feature**, which is a far better use of that information than throwing rows away (see Topic 4).
- Con: 540 rows is a small dataset. Deep trees are out; we move to shallow XGBoost with strong regularization, or regularized linear/GP models. **This is not a downside — it is the honest problem statement finally becoming visible.**
- Con: requires knowing the lag (Topic 3) — but we need that anyway.

Path D — Two-resolution hybrid. ★ Recognize that we have **two different targets at two resolutions**:

- Process models ($\text{MV} \rightarrow \text{CV}$): CVs are *online sensors* at 1-min resolution. No replication problem at all. Train these on the full high-frequency dataset.
- Quality model ($\text{CV} + \text{DV} \rightarrow \text{PSI200}$): if PSI200 is a lab value, train on the aggregated 540 rows (Path C). If PSI200 comes from an **online particle-size analyser**, there is no replication problem and Path C is unnecessary for it.
- Pro: uses each dataset at its true information content.
- Pro: this is exactly the right way to exploit the cascade structure.

Open question we must answer first

Is **PSI200** a laboratory shift value or an online analyser reading? This changes the recommendation completely. Diagnostic: count distinct consecutive values.

```
df['PSI200'].diff().ne(0).sum() # number of actual changes
df.groupby(df['PSI200'].diff().ne(0).cumsum()).size().describe() # run lengths
```

If the median run length is 1, it's an online analyser (Topic 2 mostly dissolves). If it's 480, it's shift lab data and Path C/D is mandatory.

Recommendation

Run the diagnostic first. Then **Path D**, with Path C applied to the quality model if the target proves to be low-frequency. Implement **Path A immediately** regardless, as a permanent metric alongside the current one.

Topic 3 — Dead Time and Temporal Alignment

What the code does today

`create_segmented_dataset` in `core/segmentation.py` slices rows and keeps every column at the same index:

```
segment_df = df.iloc[instance.start:instance.end].copy()
```

So **Ore** at 10:00 is paired with **PSI200** at 10:00. There is **no lag anywhere in the feature pipeline**. Interestingly, `analysis/analyzer.py` *does* estimate lags via cross-correlation and writes them to the analysis CSVs — but nothing consumes them.

The ML concept, explained simply

Dead time (transport delay) is the time between a cause and its observable effect. Every physical process has it.

In a grinding circuit:


```
Ore feed change —(1-3 min: belt transport)→▶ Mill inlet
Mill inlet      —(5-15 min: mill residence)→▶ Mill discharge / sump
Sump           —(2-5 min: pump + cyclone)→▶ DensityHC / PressureHC
Cyclone overflow —(1-5 min: sampler + analyser)→▶ PSI200 reading
Total: ~10-30 min
```

If you feed a model $Ore(t)$ and ask it to predict $PSI200(t)$, you are asking it to explain an effect using a cause that hadn't propagated yet. What the model actually latches onto is **autocorrelation**: $Ore(t) \approx Ore(t-20)$, so it partially works — which is exactly why the mistake survives. But it is a diluted, blurred version of the true relationship.

Numeric intuition: if the true relation is $PSI200(t) = f(Ore(t-20))$ and $corr(Ore(t), Ore(t-20)) = 0.7$, then aligning at lag 0 caps your achievable correlation at roughly $0.7 \times$ the true one. You leave $\sim 30\%$ of the signal on the table and can never recover it by tuning.

Why it matters here

Two extra consequences specific to our setup:

1. **The motif constraints are misaligned too.** The density pattern looks for "stable WaterZumpf while Ore/WaterMill vary" and then measures the **DensityHC** response *in the same window*. If the density response lands 8 minutes later, part of it falls outside the 60-minute window's meaningful region — and the correlation filter (`correlation_rules, min_correlation_strength: 0.1`) is evaluating misaligned series. We may be discarding good motifs and keeping bad ones.
2. **The cascade compounds the misalignment.** Stage 1 is misaligned by the MV→CV lag, stage 2 by the CV→PSI200 lag. The errors do not cancel.

The candidate paths

Path A — Single global lag, grid-searched. Shift all features by a constant L , try $L \in \{0, 5, 10, \dots, 45\}$ min, pick the best validation MAE.

- Pro: dead simple, one parameter, big expected gain.
- Con: forces one lag on variables that genuinely have different lags (WaterZumpf → PressureHC is fast; Ore → PSI200 is slow).

Path B — Per-pair lag from cross-correlation. ★ For each (input, output) pair, compute the lag maximizing cross-correlation. We already have this code in `analysis/analyzer.py` — we just need to *use* its output.

- Pro: physically faithful; reuses existing logic.
- Pro: the estimated lags are themselves a **diagnostic deliverable** — process engineers can sanity-check them ("15 min mill residence, yes that's right").
- Con: cross-correlation is fooled by common drift/trends. Must detrend or difference first.
- Con: lags vary with throughput (higher Ore → shorter residence). A fixed lag is an approximation.

Path C — Rolling-window aggregation instead of point lags. ★★ Rather than "the value 20 minutes ago", use "the mean/std/slope over minutes t-35 to t-5".

- Pro: **robust to lag misestimation**. If the true lag is 18 and your window covers 5-35, you captured it. A point lag of 20 vs. a true 18 misses.
- Pro: naturally denoises (a 30-min mean over a noisy sensor is far more stable than one sample), which partly substitutes for Topic 1's smoothing.
- Pro: composes perfectly with Topic 2 Path C — the aggregation window *is* the residence window.
- Pro: the `std` and `slope` aggregates carry information a point lag simply cannot express.
- Con: more features (feature count $\approx n_vars \times n_aggregates$); needs regularization or feature selection, especially with 540 rows.

Path D — Flow-normalized (throughput-varying) lag. Compute lag dynamically as `residence_volume / current_flow`.

- Pro: most physically correct; handles the throughput dependency.
- Con: needs mill/sump volume estimates; a lot of engineering for a second-order refinement.
- Verdict: park it. Revisit only if Path C plateaus.

Path E — Let a sequence model learn the alignment. Feed the raw `[t-60, t]` multivariate window to a 1D-CNN or GRU.

- Pro: no manual lag engineering; learns lags, shapes, and interactions jointly.
- Pro: we have plenty of 1-min data for the *process* models.
- Con: needs far more labels than 540 for the quality model — viable only if PSI200 is online.

- Con: loses the interpretability that makes this model acceptable to plant staff.
- Verdict: strong candidate for the MV→CV stage later; not for stage one.

Recommendation

Path B to estimate the lags (and publish them for engineer review), then Path C to actually build features. Use the estimated lag to *centre* the aggregation window rather than to shift a point value. This is the highest expected-value change in the whole document after fixing the leaks.

Topic 4 — Motif Selection vs. Steady-State Detection

This is the conceptual heart of the project, so it gets the longest treatment.

What the code does today

The pipeline runs several STUMPY-based pattern discoveries (*mv*, *density*, *inverse*, *dynamic*, optionally *pressure*), each computing a multivariate matrix profile and greedily extracting motifs:

```
# patterns/mv_pattern.py:69-70
matrix_profile, profile_indices = stumpy.mstump(T, m=self.window_size)
mp_distances = np.sqrt(np.mean(matrix_profile**2, axis=0))
```

It then keeps only rows inside the discovered 60-minute windows, merges patterns, dedups overlaps, and writes *segmented_motifs_all_XX.csv*. Everything outside a motif is discarded.

Layered on top there are **four more filters**:

- *filter_thresholds*: *Ore* $\in (130, 220)$, *PulpHC* $\in (350, 600)$, *DensityHC* $\in (1600, 1920)$
- *filter_data_adaptive*: rolling median $\pm 5 \cdot \text{MAD}$
- $16 < \text{PSI}_{200} < 30$ in *train_models.py*:282
- *target_clip_min/max* = [15, 30] on the predictions

The ML concept, explained simply

There are two very different reasons to select a subset of your data, and we have conflated them:

(1) Removing corrupt data. Sensor dropouts, frozen transmitters, mill stopped, calibration excursions. These rows contain *no valid information*. Deleting them is unambiguously correct.

(2) Selecting "informative" operating conditions. Keeping only steady, repeating windows. This is **not** data cleaning — it is changing the distribution you train on. And that has a formal name: **covariate shift**, deliberately induced.

A motif is defined as *a subsequence that repeats*. Ask what that actually selects for:

Repeating patterns in a mill are the operating points the plant sits at **most often** — i.e. the *normal* operating envelope, at its *most typical* settings.

Now consider what we want the model for: an **optimizer** that proposes setpoints. The optimizer's entire job is to suggest something *different from what the plant normally does*. We have trained the model exclusively on "what the plant normally does" and are about to ask it about everything else.

Analogy: you want to advise a driver on the best speed for fuel economy, so you collect data — but you only keep the segments where they were driving steadily at their habitual 100 km/h. Your model will be superb at predicting fuel use at 100 km/h and will have literally no idea what happens at 80 or 130. Yet 80 or 130 is the entire question.

The information-theoretic view

To learn $y = f(x)$, you need **variance in x**. Statistically, the variance of a regression coefficient scales as $1 / \text{Var}(x)$. Motif extraction, by construction, *reduces* $\text{Var}(x)$ for the very variables we manipulate. The density pattern is explicit about it:

stable WaterZumpf ($\text{CV} \leq 1\%$), varying Ore/WaterMill ($\text{CV} \geq 0.08\%, \geq 0.15\%$)

A "varying" threshold of **CV $\geq 0.08\%$** is essentially "not perfectly frozen". We are selecting windows where Ore barely moves and asking the model to learn Ore's effect.

Meanwhile, the transitions we throw away — the step changes, the grade changes, the operator interventions — are precisely the windows with the **highest information content** about causal gains. In system identification this is elementary: you identify a plant by *exciting* it, not by watching it sit still. A PRBS test signal exists for exactly this reason.

But the motif approach isn't wrong — it's answering a different question

To be fair to the current design, motifs do deliver two real benefits:

1. **Noise suppression by averaging.** Steady windows have high SNR. In a genuinely filthy historian, that matters.
2. **Quasi-steady-state pairing.** During a transient, `PSI200(t)` does not correspond to `Ore(t)` at all (Topic 3). In steady state, the lag problem *vanishes* — everything has settled, so any alignment is correct. **Motif extraction is, in effect, an implicit and rather expensive workaround for not having solved dead time.**

That second point is important: once we handle lag properly (Topic 3), one of the two main justifications for aggressive motif filtering evaporates.

The candidate paths

Path A — Status quo: hard motif gate.

- Pro: high SNR; already built and working.
- Con: narrow training distribution; tree models cannot extrapolate outside it; unusable for optimization beyond the habitual envelope. Discards the most informative data.
- Con: four stacked filters make the *effective* training region opaque. Nobody currently knows what fraction of the plant's operating envelope survives.

Path B — Keep all data, add a `quality_weight` column. ★★ Replace the hard gate with a continuous score per row, used as `sample_weight`:

```
quality_weight = w_valid × w_steady × w_recent
w_valid   : 0 if sensor invalid / mill down / out of physical range, else 1
w_steady  : e.g.  $\exp(-k \cdot \text{rolling\_CV\_of\_MVs})$  → steady windows count more
w_recent  : optional decay so recent operation counts more (handles drift)
```

- Pro: **you get the SNR benefit without the distribution narrowing**. A transient row still contributes — just less.
- Pro: nothing is silently discarded; the weight is inspectable and tunable as one number.
- Pro: it converts a hard, brittle, multi-threshold decision into a smooth one. Small changes in a threshold no longer flip thousands of rows in or out.
- Con: XGBoost weighting is easy, but choosing **k** needs validation.

Path C — Steady-state detection instead of motif matching. ★ Classic process-industry approach: flag steady state via rolling standard deviation, a Student's t-test on window slope, or the F-like ratio test (Cao & Rhinehart). Keep steady windows, but *do not require them to repeat*.

- Pro: much cheaper than a matrix profile ($O(n)$ vs. the mstump cost).
- Pro: keeps **unique** steady operating points that motif matching rejects for lack of a partner. Those unique points are the ones extending your coverage — exactly what the optimizer needs.
- Pro: far easier to explain to process engineers than "multivariate matrix profile radius".
- Con: loses the shape-matching that makes motifs elegant.

Path D — Motifs relegated to a diagnostic / feature role. ★ Stop using motifs to *filter rows*. Instead:

- Use **pattern_type** and **motif_id** as **metadata** for grouped cross-validation (Topic 7).
- Use the matrix-profile distance as an **anomaly score feature** (high distance = unusual operation = probably lower-confidence label).
- Keep the motif analysis outputs as the **process-insight deliverable** they genuinely are (the density/inverse/dynamic analyses are legitimately interesting to engineers).
- Pro: keeps all the value already built, removes the harmful part.
- Pro: the matrix-profile distance doubles as an out-of-domain detector (Topic 10) — a real bonus.

Path E — Actively enrich the sparse regions. Once weights replace gates, deliberately **up-weight** rare operating points (inverse propensity / density-ratio weighting) so the model spends capacity where data is thin.

- Pro: directly targets the optimizer's needs.

- Con: up-weighting rare points also up-weights their noise. Needs care and good uncertainty estimates. Advanced — phase it in later.

Path F — Plant step tests. The real answer, if it is ever politically possible: run deliberate step changes to build a proper identification dataset.

- Pro: would resolve most of this document at a stroke.
- Con: production cost, requires operations buy-in. Worth *asking* about; even 2 days of gentle steps would be transformative. Document the request.

Recommendation

Path B + Path D together, with Path C's detector supplying the w_{steady} term.

Concretely: keep the full filtered-for-validity dataset, compute a steady-state score, use it as a sample weight, and demote motifs from gatekeeper to metadata/diagnostic/anomaly-score. Then measure whether the widened training distribution actually helps — using a test set that deliberately includes operating points *outside* the motif envelope, because that is the scenario we actually care about.

Key experiment to run: train Model-A on motifs only and Model-B on all-data-with-weights. Evaluate both on (i) motif-like test rows and (ii) non-motif test rows. My prediction: A wins slightly on (i) and loses badly on (ii). If so, the case is settled.

Topic 5 — Variable Taxonomy (MV / CV / DV) and Model Invertibility

What the code does today

```
# data_preparation/config/defaults.py:20-23
mv_features = ['Ore', 'WaterMill', 'WaterZumpf']
cv_features = ['DensityHC', 'PulpHC', 'PressureHC', 'CirculativeLoad']
dv_features = ['Class_15', 'Daiki', 'FE', 'MotorAmp']
target      = 'PSI200'
```

The ML concept, explained simply

In process control the three categories have precise, operationally meaningful definitions:

Class	Definition	Test question	At optimization time
MV	Manipulated variable — the operator/controller <i>sets</i> it	"Can I turn a knob and change this?"	Free — this is what we solve for
CV	Controlled variable — a <i>measured response</i> of the plant	"Does this change because the plant reacted?"	Predicted by the model
DV	Disturbance variable — measured but <i>not</i> controllable	"Does this arrive from outside, independent of my knobs?"	Fixed at its current/forecast value

The distinction is not cosmetic. It determines **what you can solve for**. An optimizer does:

```
maximize/target  PSI200
over             MV  (free variables)
given           DV  (known constants)
subject to      CV = f_process(MV, DV)  and CV within limits
```

If you misclassify a variable, this optimization becomes ill-posed.

The specific problems in our config

Problem 1: **MotorAmp** is labelled a DV.

Motor amperage is a *response* to mill charge, ore hardness, and water addition. It is not something arriving from outside the plant, and it is not something you set.

Consequences:

- At optimization time, you must supply a value for every DV. What **MotorAmp** do you supply? You don't know it — it depends on the MVs you're solving for. The optimization is **not well posed**.

- If you freeze it at its current value while varying **Ore**, you're telling the model "increase the ore by 30 t/h and the motor draw stays identical" — a physically impossible input combination. The model will happily return a confident, meaningless number.
- It is also a strong predictor (it correlates with mill load and thus with grind), so it probably has high feature importance — which makes the model look *better* offline while making it *less* usable. The worst kind of bug.

Note **config.py:23** has a commented-out line placing **MotorAmp** in the MVs. Either home is defensible (MV if there is a mill-speed/charge control; CV otherwise) — but DV is not.

Problem 2: **CirculativeLoad** is an algebraic function of other CVs.

Per **calculate_circulative_load**, it is derived deterministically from **Ore**, **PulpHC**, **DensityHC**:

$$C_v \rightarrow C_m \rightarrow M_{solid_to_cyclone} \rightarrow CL = (M_{solid} - Ore) / Ore$$

So:

- Training **MV** → **CirculativeLoad** as a fourth "process model" is training a model to predict a known formula of two things we already predict. It adds a **fourth independent error source where there should be zero**.
- Feeding **CirculativeLoad** into the quality model *alongside* **PulpHC** and **DensityHC** adds no information — it is a deterministic transform of features already present. It does add **multicollinearity**, which scrambles feature importances and makes the model's internal attribution unreliable.
- The one legitimate argument for keeping it: it's a *useful non-linear transform* that trees would need many splits to construct themselves. That is real — feature engineering of known physics is good practice. But that argues for it as a **quality-model feature only**, never as a process-model target.

Problem 3: Are **Class_15**, **Daiki**, **FE** genuinely available at decision time?

These are ore-quality assays. They are true DVs conceptually (you can't control the orebody), but if they arrive from the lab *after* the shift, then at 10:00 you do not have the 10:00 assay. Training on them is fine; *deploying* on them requires the most recent available assay, which may be 8 hours stale. That train/serve gap must be simulated during validation — use the lagged-availability value in training too.

The candidate paths

Path A — Minimal correction. Move **MotorAmp** to CV; drop **CirculativeLoad** from **cv_features** (process targets) but keep it as a quality-model input.

- Pro: small diff, immediately makes the cascade invertible.
- Con: **MotorAmp** becomes a fourth process model to train — more cascade error. (Mitigated by the fact that it *should* be easy to predict from MVs.)

Path B — Minimal CV set. Reduce CVs to the physically independent measurements: **['DensityHC', 'PulpHC', 'PressureHC']**, and compute **CirculativeLoad** *from the predicted CVs* inside the cascade rather than modelling it.

- Pro: cleanest. Zero redundant models, physics preserved exactly, no collinearity from a separately-predicted CL that is inconsistent with the predicted PulpHC/DensityHC.
- Pro: the CL fed to the quality model is then guaranteed *consistent* with the other CVs. In the current design, predicted-CL and predicted-PulpHC can contradict each other.
- Con: requires a small code path to recompute CL mid-cascade.
- **This is the correct design.**

Path C — Availability-aware DVs. Add an explicit **available_at** concept: each DV is represented by its most recent value *as of* the prediction time, plus an **age** feature.

- Pro: eliminates a whole class of silent train/serve skew.
- Con: extra plumbing; only worth it once we confirm the assays are delayed.

Path D — Re-derive the taxonomy with a process engineer. Sit down and classify every tag.

- Pro: costs an hour; prevents months of subtly-wrong models.
- **Do this regardless of which technical path we take.**

Recommendation

Path B + Path D now; Path C after confirming assay latency. Also add a runtime assertion: no variable may appear in more than one of MV/CV/DV, and every DV must be justifiable as "knowable and uncontrollable at decision time".

Topic 6 — Cascade Coupling and Error Compounding

What the code does today

```
# modeling/config.py:128
quality_model_use_predicted_cv: bool = True
```

When true, `_predict_cv_features` runs the trained process models over the training MVs and feeds those predictions — not the measured CVs — into the quality model. The code already measures the consequence honestly (`train_models.py:509-528`), comparing predicted-CV vs. actual-CV cascade performance. That diagnostic is genuinely good practice.

The ML concept, explained simply

A cascade is $MV \rightarrow CV \rightarrow y$. There are two ways to train stage 2, and they fail differently.

Teacher forcing (train stage 2 on *measured* CVs): stage 2 sees clean, true inputs. But at inference it receives stage 1's noisy predictions — inputs from a distribution it never saw. This is **exposure bias**, exactly the problem sequence models have with teacher forcing. Small stage-1 errors get amplified because stage 2 was never taught to be robust to them.

Predicted-input training (current setting): stage 2 sees the same noisy inputs it will see in production. Train and serve match. But there is a subtle and serious side effect:

Predicted CVs are a **deterministic function of the MVs**. $CV_{pred} = g(MV)$, so $X_{quality} = [g_1(MV), g_2(MV), g_3(MV), g_4(MV), DV]$.

Every CV column is now a transform of the same 3 numbers. The quality model has effectively become $f(MV, DV)$ — **the cascade has collapsed into a single flat model wearing a cascade costume**. Concretely:

- Feature importances are meaningless: the four CV columns split ~arbitrarily between themselves because they're mutually redundant functions of the same

inputs.

- The CV layer stops carrying physical meaning, so CV-based operating constraints (e.g. "keep PressureHC below X") lose their grounding.
- Stage 1's *systematic bias* gets baked in. If the DensityHC model reads 15 units low, the quality model learns to compensate — until you retrain stage 1 and the compensation becomes wrong. **The two stages are now silently coupled and must always be retrained together.**

Why do we even have a cascade?

Worth stating plainly, because if we can't answer this, flat is better:

1. **Constraints live on CVs.** The optimizer must respect pressure/density limits. A flat model cannot express them.
2. **CVs are observable online.** In production you can compare predicted vs. measured CV and detect model drift *before* it corrupts the PSI200 prediction. Invaluable for monitoring.
3. **CVs have abundant labels** (1-min sensors) while PSI200 may not (Topic 2). The cascade lets the data-rich stage do the heavy lifting.
4. **Interpretability/trust.** Engineers can validate "Ore +10 → Density +X" independently.

All four reasons are real. **The cascade is the right architecture** — we just need to stop undermining reason 1 and 4 with the predicted-CV coupling.

The candidate paths

Path A — Teacher forcing (`use_predicted_cv = False`).

- Pro: clean physical meaning, decoupled stages, honest feature importances.
- Con: exposure bias — cascade R^2 will be worse than the sum of its parts.

Path B — Noise-augmented training on actual CVs. ★★ Train on measured CVs, but inject Gaussian noise matched to each process model's residual distribution:

`CV_train = CV_actual +  $\epsilon$ ,  $\epsilon \sim N(\text{bias}_i, \sigma_i)$` from stage-1 out-of-fold residuals. Optionally generate several noisy copies per row (a form of data augmentation).

- Pro: gets the robustness of predicted-CV training **without** collapsing CVs into functions of MV — the CV columns retain their independent, physically-meaningful variation.
- Pro: it's a **regularizer** (input noise $\approx$ ridge penalty), which helps with our small effective sample size.
- Pro: stages stay decoupled — retrain one without invalidating the other.
- Con: needs out-of-fold stage-1 residuals to get σ right (in-sample residuals under-state it).

Path C — Residual-corrected cascade. Train stage 2 on actual CVs, then fit a small correction model on the cascade's end-to-end residuals.

- Pro: keeps stage 2 pristine; the correction absorbs compounding.
- Con: a third model to maintain and explain.

Path D — Joint / end-to-end training. Optimize both stages against the final PSI200 loss plus an auxiliary CV loss ($L = L_{\text{PSI200}} + \lambda \cdot L_{\text{CV}}$).

- Pro: theoretically optimal balance between the two objectives; λ tunes the trade-off.
- Con: XGBoost can't do this — needs a differentiable framework (PyTorch MLPs, or GPs).
- Con: significant rewrite. Only if B and C both disappoint.

Path E — Keep both and let the diagnostic decide. Retain the existing predicted-vs-actual comparison as a *permanent* CI metric, and report three numbers every run:

1. Stage-2 with actual CVs (the ceiling)
2. Full cascade with predicted CVs (reality)
3. The delta (the cost of stage 1)

- **Do this regardless.** It's already 80% implemented and it's the only way to see whether any of A-D actually helped.

Recommendation

Path B, with Path E's three-number report as the permanent scoreboard. Keep the `quality_model_use_predicted_cv` flag so we can A/B it, but add a third option (`'actual' | 'predicted' | 'noise_augmented'`) rather than a boolean.

Also: **stage 1's residual σ per CV is a deliverable in its own right.** If the DensityHC process model has an R^2 of 0.4, no amount of stage-2 cleverness will save the cascade, and we should know that number prominently.

Topic 7 — Validation Strategy Under Autocorrelation

What the code does today

```
# modeling/train_models.py:188-190
train_size = int(len(self.df) * (1 - self.config.model.test_size))
train_df = self.df.iloc[:train_size]
test_df = self.df.iloc[train_size:]
```

with an inner `TimeSeriesSplit(n_splits=5)` for `GridSearchCV`, and cascade validation drawing a random sample:

```
# modeling/train_models.py:455
sample_indices = np.random.choice(test_df.index, size=n_samples, replace=False)
```

There is a good defensive fix already in place — the chronological re-sort at `train_models.py:160-163` — showing the leakage risk was recognized. But it doesn't go far enough.

The ML concept, explained simply

Cross-validation estimates generalization by holding out data the model hasn't seen. It only works if held-out rows are **independent** of training rows. With time-series process data, consecutive rows are nearly identical, so "unseen" is a fiction.

Three distinct dependency structures exist in our data, and each needs its own defence:

(1) Temporal autocorrelation. `Ore(t) ≈ Ore(t+1)`. The row after the split boundary is almost the same as the row before it. Fix: a **purge gap** — drop a buffer ($\geq$ the

longest lag/window, so ≥ 60 min) between train and test.

(2) Motif segment membership. A 60-minute motif instance produces 60 rows that are, by construction, a smooth trajectory. If 40 land in train and 20 in test, the test rows are almost memorized. Fix: **group by** `motif_id / segment_start` so a segment is wholly in one side.

(3) Label replication. All rows sharing one filled lab value (Topic 2). Fix: **group by** `lab-sample id`.

Concrete illustration of how badly this inflates scores. Take a pure-noise target and a smoothed feature; with row-wise splitting on autocorrelated data you can easily measure $R^2 \approx 0.6$ on a relationship that does not exist. That is not hypothetical — it is the default failure mode of naive CV on process data.

An additional, easily-missed problem

The motifs are shuffled and merged before the CSV is written

(`merge_motif_collections(..., shuffle=True)`), and the CSV is built pattern-by-pattern. `train_models.py` re-sorts by `TimeStamp`, which fixes ordering — but the resulting series is **non-contiguous in time**: row i and row $i+1$ may be days apart, while rows from the same segment are scattered across the whole file. `TimeSeriesSplit`'s assumption of a contiguous timeline no longer holds, and a purge gap measured in *rows* means nothing.

Also, the quality model re-splits after filtering:

```
# modeling/train_models.py:279-288
combined_df = pd.concat([train_df, test_df], ignore_index=False)
... target_mask ...
split_idx = int(len(combined_df_filtered) * (1 - test_size))
```

This is not leakage per se (the order is preserved), but it means the process models and the quality model are validated on **different test sets**, so their metrics aren't comparable and the cascade number mixes rows some stages trained on.

The candidate paths

Path A — Purged, grouped, time-blocked CV. ★★ Use

`sklearn.model_selection.GroupKFold` with time-ordered blocks, or a custom `PurgedGroupTimeSeriesSplit`:

- Groups = lab-sample id (or `motif_id` if no lab grouping applies)
- Blocks are contiguous in *wall-clock time*, not row index
- A purge gap of $\geq \max(\text{window}, \text{lag})$ between train and validation
- Pro: the only structurally correct option. Every other improvement is unmeasurable without it.
- Con: needs a custom splitter (~40 lines) and a genuine time column, not row position.

Path B — Hold out whole calendar periods. Train on months 1-4, validate on month 5, test on month 6.

- Pro: brutally simple, obviously honest, and mirrors deployment (you always predict the future from the past).
- Pro: also exposes **concept drift** — if month-6 performance collapses, the plant changed and you need periodic retraining. That is vital operational knowledge we currently have no view of.
- Con: only one test period; the metric is noisy. Mitigate with rolling-origin evaluation (train 1-4/test 5, train 1-5/test 6, ...) and average.

Path C — Nested CV. Outer loop for honest performance estimation, inner loop for hyperparameters.

- Pro: removes the optimism from selecting hyperparameters on the same folds you report.
- Con: multiplies compute; with Topic 8's cheaper search it becomes affordable.

Path D — Report a naive baseline alongside everything. Always print the score of:

- predicting the training mean
- predicting the previous lab value (persistence)
- a plain ridge regression on the raw features
- Pro: instantly reveals whether the XGBoost cascade is earning its complexity.
- Pro: **persistence is a genuinely strong baseline** for PSI200 and is often not beaten. If we can't beat it, that's the most important finding we could produce.
- Con: none. Do it.

Recommendation

Path A for model selection, Path B as the headline number, Path D always. Also seed the cascade-validation sampler and evaluate on the *entire* test set rather than 200 random rows — there is no compute reason to subsample.

Unify the test set across all stages: split once, up front, and pass the same masks to every model.

Topic 8 — Hyperparameter Search and the Noise Floor

What the code does today

```
# modeling/config.py:109-115
param_grid = {
    "n_estimators": [150, 300, 400],
    "learning_rate": [0.01, 0.05, 0.1],
    "max_depth": [3, 5, 8],
    "subsample": [0.6, 0.8, 0.9],
    "colsample_bytree": [0.6, 0.8, 1.0],
}
```

$3^5 = 243$ combinations $\times$ 5 folds = 1,215 fits per model, $\times$ 5 models = ~6,000 fits per run, selecting on `neg_mean_absolute_error`.

The ML concept, explained simply

Hyperparameter search picks the configuration with the best validation score. But validation scores are **random variables** — they have their own standard error. If the true difference between two configurations is 0.01 MAE and the standard error of your estimate is 0.05, you're picking the winner of a coin flip.

Worse, searching 243 candidates is 243 chances to get lucky. This is **selection bias / multiple comparisons**: the *winner's* validation score is biased optimistic by roughly the maximum of 243 noise draws. With correlated folds (Topic 7) the effective noise is even larger.

The rule of thumb: you can meaningfully distinguish configurations only when the performance gap exceeds the standard error of your validation estimate. With ~100

independent labels in a validation fold (Topic 2), that standard error is large. **We are almost certainly tuning noise.**

Why it matters here

Three concrete costs:

1. **Compute.** 6,000 fits per run makes iteration slow, which discourages the experiments we actually need (Topics 1-4 each require a re-baseline).
2. **False precision.** `training_results.json` reports "best parameters" that would change completely on a different random seed. It reads as knowledge; it isn't.
3. **Missed regularization.** The grid searches `n_estimators`, `learning_rate`, `max_depth`, `subsample`, `colsample_bytree` — but **not** `reg_alpha`, `reg_lambda`, `min_child_weight`, or `gamma`. For a small-effective-N, high-noise problem, those are the parameters that matter most. We're searching hard in the wrong subspace.

Note also `n_estimators` is being grid-searched, which is wasteful — it is the one hyperparameter you get for free via early stopping.

The candidate paths

Path A — Randomized search. `RandomizedSearchCV(n_iter=40)` over continuous distributions.

- Pro: ~30× cheaper; provably finds near-optimal configurations for far fewer trials than grid search when only a few hyperparameters actually matter (which is the usual case).
- Pro: samples continuous values instead of 3 arbitrary points.
- Con: no built-in early-stopping of unpromising trials.

Path B — Bayesian optimization (Optuna). ★

- Pro: focuses trials where they help; supports pruning (kill bad trials early).
- Pro: gives **parameter importance** output — tells us which hyperparameters even matter, which directly answers "are we tuning noise?".
- Con: an extra dependency; overkill if the model is small.

Path C — Early stopping + a short, regularization-focused search. ★★

- Fix `learning_rate = 0.03`, `n_estimators = 2000`, `early_stopping_rounds = 50` — this tunes tree count exactly and for free.
- Search only: `max_depth ∈ [2..6]`, `min_child_weight`, `reg_lambda`, `subsample`, `colsample_bytree`.
- Pro: dramatically cheaper *and* targets the parameters that control overfitting on small, noisy data.
- Pro: `max_depth` capped at ~4 is often the single best change for this kind of dataset — deep trees memorize autocorrelated segments.
- Con: needs a proper validation set for early stopping (which Topic 7 gives us).

Path D — Report the selection uncertainty. Print the mean $\pm$ std of the CV score for the top-5 configurations.

- Pro: makes it immediately visible when the top 5 are statistically indistinguishable, and then we should prefer the **simplest** (shallowest, most regularized) among them — a one-standard-error rule.
- Con: none.

Path E — Question whether XGBoost is the right model at all. With ~540 effective samples and ~12 features in a smooth physical process, gradient boosting is not obviously the best choice. Alternatives:

- **Regularized linear / spline models (GAM):** smooth, extrapolate sanely, fully interpretable, perfect for small N. Often within a few percent of XGBoost on process data.
- **Gaussian Process:** we already have `gp_modelling/`. Native uncertainty, excellent at small N, smooth extrapolation. The $O(n^3)$ cost is irrelevant at $n=540$ — **the small dataset that hurts XGBoost is exactly the regime where GPs shine.**
- Verdict: the existing GP work may be more valuable than it currently looks. Benchmark it properly once Topics 1-3 are fixed.

Recommendation

Path C + Path D immediately (cheap, strictly better). **Path B** once the pipeline is stable. And run **Path E's benchmark** — XGBoost vs. GAM vs. GP on the corrected dataset — as a one-off study before committing further to trees.

Topic 9 — Extrapolation, Physics Constraints, and Hybrid Models

What the code does today

Trees, plus hard clipping:

```
# modeling/config.py:134-135
target_clip_min: float = 15.0
target_clip_max: float = 30.0
```

applied after training on $16 < \text{PSI200} < 30$.

The ML concept, explained simply

Tree ensembles cannot extrapolate. At all. A decision tree partitions the input space into boxes and predicts a constant in each. Outside the training range, the outermost box's constant is returned forever.

Example. Train on $\text{Ore} \in [130, 220]$. Ask for $\text{Ore} = 260$:

Model	Prediction at Ore=260
Linear	extrapolates along the fitted slope — possibly wrong, but <i>directionally sensible</i>
XGBoost	exactly the same value as at Ore=220 — flat, forever
GP	reverts to the prior mean with a large uncertainty band — honestly says "I don't know"

Now combine that with Topic 4: our training range is *already* narrowed by motif selection. An optimizer searching for the best **Ore** will see a flat response above the training max and may confidently recommend the boundary value — or anything beyond it, since the objective is flat there. This is precisely how model-based optimizers produce absurd setpoints.

The clipping makes it worse, not better: it converts "wrong prediction" into "wrong prediction that looks plausible". The model returns 30.0 with no signal that it has no

idea. **Clipping** is suppressing a symptom that should instead trigger a refusal.

The ML concept: physics as inductive bias

When data is scarce and noisy, **prior knowledge is worth more than more model capacity**. We know things about grinding:

- More ore at constant power → coarser grind (PSI200 ↑, monotone)
- More water in mill → lower density → typically finer at the cyclone, up to a point
- Higher circulating load → generally finer product
- Cyclone pressure ↑ → finer cut

Encoding these turns unconstrained curve-fitting into constrained curve-fitting, which (a) reduces variance, (b) makes extrapolation sane, and (c) makes the model *defensible* to process engineers — which is what actually gets it deployed.

The candidate paths

Path A — Monotonic constraints in XGBoost. ★★ `monotone_constraints=(1, -1, 0, ...)` per feature.

- Pro: one config line. XGBoost enforces it exactly during tree construction.
- Pro: acts as a strong regularizer — it forbids the model from fitting noise-driven non-monotone wiggles, which is a large share of what it currently overfits.
- Pro: guarantees the optimizer sees a well-behaved response surface.
- Con: requires agreeing the sign for each feature (a conversation with the plant, which we want to have anyway — Topic 5 Path D).
- Con: wrong on any genuinely non-monotone relationship (water addition likely has an optimum). Only constrain the ones we're confident about; leave the rest at 0.

Path B — Hybrid / residual modeling. ★ Fit a first-principles model (Bond work index, a population-balance or empirical grinding model) as $\hat{y}_{\text{phys}}$, then train ML on the residual $y - \hat{y}_{\text{phys}}$.

- Pro: extrapolation is governed by physics, and ML only corrects local, well-sampled deviations. Far safer outside the training envelope.
- Pro: residuals are typically smaller and more homoscedastic — an easier learning problem.

- Pro: **the residual itself becomes a monitoring signal**: a drifting residual means the plant has changed (liner wear, ore change).
- Con: requires a credible physical model and its parameters. Real work, needs domain input.
- Con: a bad physical model actively hurts.

Path C — Replace clipping with abstention. ★ ★ Delete `target_clip_*`. Instead compute a domain-membership score and return `(prediction, interval, in_domain_flag)`. Outside the domain: widen the interval, flag it, and let the consumer decide.

- Pro: honest. A flagged "don't know" is enormously more useful to an optimizer than a silently clamped number.
- Pro: dovetails with Topic 10.
- Con: the downstream consumer must handle the flag.

Path D — Smooth, extrapolating model classes. GAM / spline / GP / regularized polynomial (the `poly_modelling/` folder already exists).

- Pro: sensible extrapolation by construction; monotonicity is also enforceable in GAMs.
- Pro: with ~540 effective samples this is the statistically appropriate model complexity.
- Con: may underfit genuine interactions; usually fixable with a few explicit interaction terms.

Path E — Constrain the optimizer, not the model. Add a trust-region constraint: the optimizer may only propose setpoints within (say) the 5th-95th percentile of training data, and within a limited step from the current operating point.

- Pro: solves the practical problem directly, independent of model class.
- Pro: **operationally desirable anyway** — nobody wants an advisor recommending a 40 t/h jump.
- Con: caps the achievable improvement per step (acceptable — iterate).

Recommendation

Path A + Path C + Path E now — all three are cheap and jointly eliminate the dangerous failure mode. **Path B** is the highest-ceiling option and worth scoping with a process engineer. **Path D** falls out of Topic 8's benchmark.

Topic 10 — Uncertainty Quantification and Out-of-Domain Detection

What the code does today

Point predictions only. No intervals. `gp_modelling/` produces σ but isn't part of the main pipeline.

The ML concept, explained simply

There are two fundamentally different kinds of uncertainty, and conflating them causes bad decisions:

Aleatoric (irreducible noise). The plant and the lab are noisy. Even with a perfect model and identical inputs, PSI200 varies. More data does **not** reduce this.

- Example: the same operating point sampled twice gives 22.1 and 23.4. Nothing you do fixes it.

Epistemic (model ignorance). The model hasn't seen this region of input space. More data **does** reduce this.

- Example: the model has never seen `Ore=240`; its prediction there is a guess.

Why the distinction is decisive for us:

- **High aleatoric** → the optimizer should not chase small predicted gains; they're inside the noise. It also sets a hard ceiling on achievable R^2 . *If lab reproducibility is ± 1.5 PSI200 units and the total spread is ± 3 , then $R^2 \approx 0.75$ is a perfect model.* Chasing 0.95 is chasing noise. **We should measure this ceiling explicitly.**
- **High epistemic** → the optimizer is in unexplored territory. Either don't go there, or go there *deliberately* to learn (exploration).

Why it matters here

An optimizer without uncertainty is dangerous in a specific, predictable way: it seeks the argmax of the predicted surface, and **model error is maximal exactly where the**

model is extrapolating. Optimizers therefore systematically drive toward the model's blind spots. This is a well-known pathology, and it's the reason Bayesian optimization uses acquisition functions (which balance predicted value against uncertainty) rather than raw argmax.

The candidate paths

Path A — Quantile regression. Train three XGBoost models with

`objective='reg:quantileerror'` at $\tau = 0.1 / 0.5 / 0.9$.

- Pro: minimal change; gives asymmetric intervals; captures **heteroscedasticity** (noise that varies with operating point — very likely here, e.g. noisier at high throughput).
- Con: captures aleatoric uncertainty well, epistemic poorly — the intervals do **not** widen outside the training domain, which is the case we most need.

Path B — Conformal prediction. ★★ Wrap any model: use a calibration set to find the residual quantile, then `prediction ± q`. With *Mondrian*/localized conformal, `q` varies by region.

- Pro: **distribution-free finite-sample coverage guarantee** — if you ask for 90%, you get 90%. Almost no other method gives you that.
- Pro: model-agnostic; works on the existing XGBoost with no retraining.
- Pro: cheap to implement (~30 lines).
- Con: needs a clean calibration split; grouped/purged splitting (Topic 7) is a prerequisite.
- Con: vanilla conformal gives constant-width intervals; needs the localized variant to widen out-of-domain.

Path C — Gaussian Process. ★ Already prototyped in `gp_modelling/`.

- Pro: principled epistemic uncertainty that **automatically widens away from the data** — the exact property Path A lacks.
- Pro: at $n \approx 540$ the $O(n^3)$ cost is trivial.
- Con: aleatoric/epistemic split depends on kernel choice; needs care with the WhiteKernel.
- Con: harder to explain to stakeholders than a tree.

Path D — Explicit out-of-domain detector. ★★ Independent of the predictor, score how "seen before" an input is:

- Mahalanobis distance to the training distribution
- k-NN distance in scaled feature space
- Isolation Forest score
- **The motif matrix-profile distance we already compute** (Topic 4 Path D) — a natural, domain-specific novelty score we're currently throwing away.
- Pro: simple, interpretable, and directly replaces the clipping hack.
- Pro: composable with any predictor.

Path E — Measure the aleatoric floor empirically. ★ Find near-duplicate operating conditions in the historical data and look at the spread of their PSI200 values. That spread *is* the irreducible noise.

- Pro: tells us what R^2 is even achievable, so we know when to stop optimizing. This single number would reframe the entire project.
- Pro: cheap — a k-NN query over the historical feature matrix.
- Con: near-duplicates in observed features may still differ in unobserved ones (ore mineralogy), so it's an upper bound on the noise floor. Still highly informative.

Path F — Cascade uncertainty propagation. Propagate stage-1 σ through stage 2 (Monte Carlo: sample CVs from their predictive distributions, push each through the quality model, take the spread).

- Pro: the only way to get an honest end-to-end interval in a cascade.
- Pro: naturally quantifies the compounding discussed in Topic 6.
- Con: needs stage-1 uncertainty first (so pairs with B or C).

Recommendation

Path E first (it recalibrates all our expectations for a day's work), then **Path B + Path D** as the production mechanism, with **Path F** once both stages emit intervals. Keep **Path C** in the benchmark from Topic 8.

Topic 11 — Data Leverage: Multi-Mill Pooling and Semi-Supervised Use

What the code does today

One mill at a time — `run.py` sets `mill_number = 8`, models are written to `models/mill_6/`, `models/mill_8/`. Every model uses only rows where the target and *all* features are present:

```
# modeling/train_models.py:147
self.df = self.df.dropna(subset=required_cols)
```

The ML concept, explained simply

Transfer learning / multi-task learning. Mills 6, 7 and 8 are different machines but obey the *same physics*. Training three isolated models means each learns the shared physics independently from one-third of the data — and each learns it badly, because each has too few labels.

Pooling with a `mill_id` feature lets the model learn "grinding works like this" from all the data, and "mill 8 runs 2 units coarser" as a small per-mill correction. This is **partial pooling**, and it is almost always better than either full pooling (ignore the differences) or no pooling (what we do now) when per-group data is scarce.

The gain is largest exactly in our situation: **few labels per group, strong shared structure**.

Semi-supervised / label-efficiency. The `dropna` on `required_cols` discards a row if *any* required column is missing — including the target. But the process models (`MV → CV`) don't need the target at all. If PSI200 is available for 5% of rows, we are currently training the `MV→CV` models on 5% of the data they could use, for no reason.

Why it matters here

Let's put rough numbers on it (adjust once we run the diagnostics):

Model	Rows usable today	Rows usable with fixes
MV → DensityHC	motif rows with all cols present	all valid rows, all 3 mills
MV → PressureHC	same	all valid rows, all 3 mills

Model	Rows usable today	Rows usable with fixes
CV+DV → PSI200	motif rows, one mill	all labelled samples, all 3 mills

The process models could plausibly see **10-50× more data**. And per Topic 6, stage-1 quality sets the ceiling for the whole cascade — so this is a direct route to a better final model.

The candidate paths

Path A — Pool mills with `mill_id` as a categorical feature. ★★

- Pro: 3× data with a one-line feature addition; XGBoost handles categoricals natively.
- Pro: automatically handles mills with less data (they borrow strength from the others).
- Pro: makes it possible to model a *new* mill with almost no data.
- Con: if the mills are genuinely different (different liners, ball charge, cyclone geometry), pooling can bias each. Mitigate by allowing `mill_id` to interact — trees do this naturally — and by checking per-mill residuals.
- **Test to run:** compare per-mill test MAE for pooled vs. isolated models. If pooled wins for every mill, adopt it unconditionally.

Path B — Per-mill fine-tuning. Train a pooled base model, then a small per-mill correction on its residuals.

- Pro: shared physics + mill-specific offsets, explicitly separated and inspectable.
- Pro: degrades gracefully — a mill with 20 samples gets almost pure base model.
- Con: two models per mill to maintain.

Path C — Per-target `dropna`. ★★ Drop rows only for the columns each specific model actually needs.

- Pro: trivially correct; big data gain for the process models; **zero downside**.
- Con: none. This is a straight bug fix.

Path D — Normalize away mill differences. Express features as deviations from each mill's rolling median.

- Pro: makes mills directly comparable; also neutralizes slow sensor drift.

- Con: loses absolute-level information, which matters for physical constraints. And the optimizer needs absolute setpoints, so you'd have to invert the transform carefully.

Path E — Pseudo-labelling / self-training on unlabelled rows.

- Pro: could expand the quality-model training set.
- Con: with a noisy, small-N teacher this usually amplifies the teacher's bias. **Not recommended** until the fundamentals are fixed.

Recommendation

Path C immediately (it's a bug). **Path A** with the per-mill comparison test as the decision gate. **Path B** if Path A shows mill-specific bias. Skip **Path E**.

Decision Matrix and Recommended Path

Step 0 — Five diagnostics to run before writing any modeling code

These are cheap, and several of them can *change the recommendations above*. We should not choose paths until we have these numbers.

#	Diagnostic	Question it answers	Which topics it decides
D1	Run-length distribution of PSI200 (and each DV)	Is the target an online analyser or a replicated lab value?	Topic 2 (all paths), Topic 7
D2	Cross-correlation of each MV vs. each CV, and each CV vs. PSI200 , after detrending	What are the real dead times? Are they even non-zero?	Topic 3 (all paths)

#	Diagnostic	Question it answers	Which topics it decides
D3	Coverage plot: distribution of <code>Ore/WaterMill/WaterZumpf</code> in the raw data vs. in <code>segmented_motifs_all_XX.csv</code>	How much of the operating envelope does motif filtering actually discard?	Topic 4 (A vs. B/C/D)
D4	k-NN duplicate study: for near-identical operating conditions, what is the spread of <code>PSI200</code> ?	What is the irreducible noise floor / max achievable R^2 ?	Topic 8 (E), Topic 10 (E)
D5	Missingness matrix per column, and row counts per mill	How much data does per-target <code>dropna</code> and mill pooling actually unlock?	Topic 11 (A, C)

D1, D3 and D4 are the highest-value. D1 determines whether Topic 2 is a crisis or a non-issue. D3 gives us a picture of what we're throwing away. D4 tells us what "good" even means — without it we cannot know whether $R^2=0.6$ is a failure or a near-perfect result.

Suggested location: a single `diagnostics.py` in `data_preparation/` writing a short markdown report. These are throwaway-cheap but should be committed, because we will re-run them after every structural change.

Impact vs. effort

Topic	Change	Impact	Effort	Risk	Order
1	Causal smoothing (EWMA), stop smoothing target	Critical — all metrics currently invalid	XS	Low	1
11	Per-target <code>dropna</code>	High (10-50× data for stage 1)	XS	None	1

Topic	Change	Impact	Effort	Risk	Order
7	Purged + grouped CV, unified split, seeded eval	Critical — nothing measurable without it	S	Low	1
7	Naive baselines (mean / persistence / ridge)	High (context for every number)	XS	None	1
2	ESS-corrected metrics (group by label)	High (honest reporting)	S	None	2
3	Lag estimation + windowed aggregate features	Highest R² upside	M	Med	2
2	Aggregate to label resolution (if D1 says lab)	High (structural fix)	M	Med	2
5	Fix MV/CV/DV taxonomy, CL as derived feature	High (makes cascade invertible)	S	Low	2
8	Early stopping + regularization-focused search	Med (speed + less overfit)	S	Low	2
4	Weights instead of motif gate; motifs → metadata	High (fixes optimizer usability)	M	Med	3
6	Noise-augmented CV training, 3-number scoreboard	Med	S	Low	3
9	Monotonic constraints; drop clipping	Med-High (safety + regularization)	S	Low	3
9	Trust region for the optimizer	High (operational safety)	S	Low	3
10	Conformal intervals + OOD detector	High (safe deployment)	M	Low	4
11	Multi-mill pooling	Med-High (3× data)	M	Med	4

Topic	Change	Impact	Effort	Risk	Order
8	Model benchmark: XGB vs. GAM vs. GP	Med (may simplify everything)	M	Low	4
9	Hybrid physics-residual model	Highest ceiling	L	Med	5
3	Sequence model for MV→CV	Med	L	Med	5
4	Plant step tests	Transformative	L (political)	—	ask now

Recommended phasing

Phase 0 — Diagnose (no model changes). Run D1-D5. Publish the report. Re-read this document's recommendations in light of it; several may change.

Phase 1 — Make measurement honest. Causal smoothing; per-target **dropna**; unified purged+grouped split; seeded, full-test-set cascade evaluation; naive baselines; ESS-corrected metrics. *Deliverable*: a re-baseline table, before vs. after. **Expect the numbers to get worse.** This phase produces no performance gain and is the most important phase in the plan.

Phase 2 — Fix the physics of the dataset. Lag estimation → windowed aggregate features; label-resolution aggregation if needed; corrected variable taxonomy; cheaper regularization-focused hyperparameter search. *Deliverable*: the first genuinely trustworthy model, and the first genuine improvement.

Phase 3 — Fix the training distribution and the cascade. Sample weights replacing the motif gate (with the A/B experiment from Topic 4); noise-augmented CV training; monotonic constraints; remove clipping; trust region. *Deliverable*: a model that behaves sensibly when the optimizer pushes it.

Phase 4 — Make it deployable. Conformal intervals; OOD detector; multi-mill pooling; the model-class benchmark; drift monitoring on stage-1 residuals.

Phase 5 — Raise the ceiling. Hybrid physics-residual modeling; sequence models for MV→CV; active exploration.

What "success" should look like

We should stop optimizing R^2 and start tracking a small set of numbers that actually matter:

1. **Honest test MAE** vs. the persistence baseline, on a purged, grouped, future-period split.
2. **Distance to the noise floor** from D4 — the only meaningful measure of how much headroom is left.
3. **Interval coverage** — does the 90% interval contain the truth 90% of the time?
4. **Cascade delta** — how much error the MV→CV stage adds (already instrumented).
5. **In-domain fraction** — what share of the optimizer's proposed setpoints the model can actually vouch for.

A model with $R^2=0.55$, calibrated intervals, sane monotone behaviour and an honest OOD flag is **far more valuable** than the current $R^2=0.9x$, because the first one can be trusted to drive setpoints and the second one cannot.

Things we should explicitly decide as a team

- Is **PSI200** a lab value or an online reading? (*D1 — blocks Topic 2*)
- Is **MotorAmp** an MV or a CV? (*blocks Topic 5*)
- Which MV→PSI200 relationships are we confident are monotone, and in which direction? (*blocks Topic 9 Path A*)
- What is the lab's reproducibility spec for PSI200? (*cross-check for D4*)
- Can we ever get plant step tests? (*Topic 4 Path F — worth asking even if the answer is no*)
- Do the ore assays (**Class_15**, **Daiki**, **FE**) arrive in real time or after the shift? (*blocks Topic 5 Path C*)

Summary in one paragraph

The cascade architecture is right and should be kept. The motif work is genuinely good engineering that is currently pointed at the wrong target — it should become a feature, weighting and diagnostic layer rather than a data gate. But none of that is the first problem. The first problem is that acausal smoothing, replicated labels, and row-wise splitting mean we do not currently know how good our models are, and the second problem is that missing dead-time alignment caps how good they can get. Fix

measurement, then fix alignment, then fix the training distribution — in that order. Everything else in this document is refinement on top of those three.